



Res3 - R1.02

Encapsulation, couches et modèle OSI



Présentation du module Res3

Le module Res3 est découpé en (volume horaire susceptible d'évoluer)

- ↳ 6 heures de CM.
- ↳ 4 séances de TD de 2 heures.
- ↳ 4 séances de TP de 4 heures.

L'évaluation du module Res3 sera la suivante :

- ↳ 1 QCM pour évaluer l'apprentissage des cours (semaine 49).
- ↳ 1 DS pour évaluer les notions vues en TD et en cours (semaine 2).
- ↳ Les notes de TP.



Objectifs

Les objectifs de ce module sont :

- ↳ De découvrir de quoi est constituée une trame et de comprendre les modèles utilisés dans le domaine des réseaux.
- ↳ D'étudier comment fonctionnent les protocoles UDP et TCP.

Différents points seront abordés :

- Constitution d'une trame Ethernet et d'un paquet IP.
- Les notions de couches et d'encapsulation.
- Les modèles OSI et TCP/IP.
- Utilisation des modèles en conception et en dépannage.
- Zoom sur UDP et TCP.



Sommaire

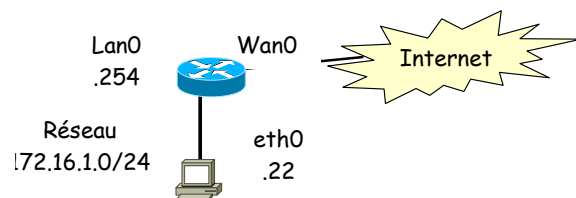
1. Constitution d'une trame Ethernet
2. Constitution d'un paquet IPv4
3. Notions d'encapsulation
4. Modèle en couches
5. Le modèle OSI
6. Le modèle TCP/IP
7. Comparaison des modèles et usages
8. Rôle de la couche transport
9. Les protocoles de la couche transport
10. Focus sur le protocole TCP
11. Exemple d'échange TCP



Constitution d'une trame Ethernet



Exemple d'observation de trames :



- Entre le moment où l'utilisateur tape "ping 172.16.1.254" dans une fenêtre de commande pour vérifier la connectivité à son routeur et l'envoi de la trame sur le réseau, plusieurs traitements successifs sont effectués, c'est ce que l'on appelle une mise en trame.
 - La machine cible reçoit cette trame, l'analyse et envoie une réponse.
- Étudions sur l'exemple suivant comment sont construites ces trames.



Constitution d'une trame Ethernet



Voici la capture de l'échange effectuée sur le PC :

No.	Time	Source	Destination	Protocol	Length	Info
3	0.000147226	172.16.1.22	172.16.1.254	ICMP	98	Echo (ping) request id=0x1ec5, seq=1/256, ttl=64 (reply in 4)
4	0.000283556	172.16.1.2...	172.16.1.22	ICMP	98	Echo (ping) reply id=0x1ec5, seq=1/256, ttl=64 (request in 3)

>	Frame 3: 98 bytes on wire (784 bits), 98 bytes captured (784 bits) on interface eth0, id 0
>	Ethernet II, Src: HewlettP_14:72:58 (d4:85:64:14:72:58), Dst: Dell_69:f2:10 (64:00:6a:69:f2:10)
>	Internet Protocol Version 4, Src: 172.16.1.22, Dst: 172.16.1.254
>	Internet Control Message Protocol
>	Type: 8 (Echo (ping) request)
>	Code: 0
>	Checksum: 0xe06d [correct]
>	[Checksum Status: Good]
>	Identifier (BE): 7877 (0x1ec5)
>	Identifier (LE): 50462 (0xc51e)
>	Sequence number (BE): 1 (0x0001)
>	Sequence number (LE): 256 (0x0100)
>	[Response frame: 4]
>	Timestamp from icmp data: Nov 2, 2022 14:25:59.000000000 Paris, Madrid
>	[Timestamp from icmp data (relative): 0.599695008 seconds]
>	Data (48 bytes)

0000	64 00 6a 69 f2 10 d4 85	64 14 72 58 08 00 45 00	d-ji---d-rX--E-
0010	00 54 c5 b7 40 00 40 01	19 bd ac 10 01 16 ac 10	-T-@-.....
0020	01 fe 08 00 e0 6d 1e c5	00 01 e7 6f 62 63 00 00	m-....obc--
0030	00 00 e7 25 09 00 00 00	00 00 10 11 12 13 14 15	...%.....
0040	16 17 18 19 1a 1b 1c 1d	1e 1f 20 21 22 23 24 25	-!""#\$%
0050	26 27 28 29 2a 2b 2c 2d	2e 2f 30 31 32 33 34 35	8'()*+,-./012345
0060	36 37		67



Constitution d'une trame Ethernet



Une analogie possible : le train

- La locomotive représente le début de la trame : c'est ce qui nous permet de la repérer.



- La première voiture s'appelle **la trame Ethernet**. Parmi les informations de cette première voiture, elle va contenir une qui décrit la seconde voiture, c'est-à-dire que contient la seconde voiture (première classe, 2^{ème} classe ou wagon restaurant).
- La seconde voiture va, entre autre contenir une information pour décrire la 3^{ème} voiture, ainsi de suite.
- Dans la dernière voiture, le contrôleur a déposé tous les documents de chargement du train afin de vérifier qu'il ne manque rien à l'arrivée ou que rien n'a été détérioré.



Constitution d'une trame Ethernet



Une analogie possible : le train - la locomotive

- La locomotive et la dernière "voiture" du train ne sont pas visibles sur Wireshark. Pourquoi ?
 - ↳ La "locomotive" s'appelle le **préambule** et sert à synchroniser le départ et l'arrivée des trames.
 - ↳ La dernière "voiture" s'appelle le **FCS** (Frame Check Sequence) et sert à contrôler l'intégrité du train.
- Wireshark :
 - ↳ Détecte le début de la trame.
 - ↳ Retire le préambule puis lit le contenu.
 - ↳ Enfin vérifie le FCS pour savoir si la trame est correcte.
- **Le préambule et le FCS ne contenant pas d'informations utiles à la machine ou à l'utilisateur, ils ne sont donc pas affichés.**





Une analogie possible : le train - la 1^{ère} voiture (capture diapo 7)

- La première voiture s'appelle **l'entête Ethernet II** (car elle est reçue par une carte réseau Ethernet) :

```
> Frame 3: 98 bytes on wire (784 bits), 98 bytes captured (784 bits) on interface eth0, id 0
> Ethernet II, Src: HewlettP_14:72:58 (d4:85:64:14:72:58), Dst: Dell_69:f2:10 (64:00:6a:69:f2:10)
  > Destination: Dell_69:f2:10 (64:00:6a:69:f2:10)
  > Source: HewlettP_14:72:58 (d4:85:64:14:72:58)
  Type: IPv4 (0x0800)
```

- Elle contient 3 champs :

- ↳ Un champ codé sur 6 octets qui contient l'adresse MAC (ou matérielle ou "hardware addresses" destination).
- ↳ Un champ codé sur 6 octets qui contient l'adresse MAC source.
- ↳ Le champ "type" qui va décrire le contenu de la voiture suivante, ici : un paquet IPv4. **Les données Ethernet vont contenir un paquet IPv4.**

- **A chaque fois qu'un paquet IP est routé par un routeur, les adresses MAC sources et destination changent.**



Une analogie possible : le train - la 2^{ème} voiture (capture diapo 7)

- Comme l'indique l'information contenue dans la 1^{ère} voiture, la 2^{ème} voiture s'appelle **l'entête IPv4** (les données Ethernet contiennent un paquet IPv4) :

- Elle contient, entre autre :

- ↳ L'adresse logique IPv4 source (la machine qui a construit le paquet IPv4, ici la machine qui a émise le ping).

- ↳ L'adresse logique IPv4 destination (la machine qui doit décoder le paquet IPv4, ici qui doit répondre au ping).

- ↳ Le champ type qui décrit le contenu de la voiture suivante, ici : un message ICMP.

- On notera la présence d'un champ intéressant le TTL qui est la durée de vie du paquet en nombre de sauts de routeurs.

- **Important : la fonction de routage ne change pas les adresses IPv4 source et destination.**

```
> Internet Protocol Version 4, Src: 172.16.1.22, Dst: 172.16.1.254
  0100 .... = Version: 4
  .... 0101 = Header Length: 20 bytes (5)
  > Differentiated Services Field: 0x00 (DSCP: CS0, ECN: Not-ECT)
  Total Length: 84
  Identification: 0xc5b7 (50615)
  > Flags: 0x4000, Don't fragment
  Fragment offset: 0
  Time to live: 64
  Protocol: ICMP (1)
  Header checksum: 0x19bd [validation disabled]
  [Header checksum status: Unverified]
  Source: 172.16.1.22
  Destination: 172.16.1.254
```



Une analogie possible : le train - la 3^{ème} voiture (capture diapo 7)

- Comme l'indique l'information contenue dans la 2^{ème} voiture, la 3^{ème} voiture s'appelle un **message ICMP** (une entête ICMP et des données ICMP) :

- Elle contient :

- ↳ Le type de requête, ici un ping request.

- ↳ Un identifiant (pour savoir à quelle commande ping sur la machine fait référence ce message).

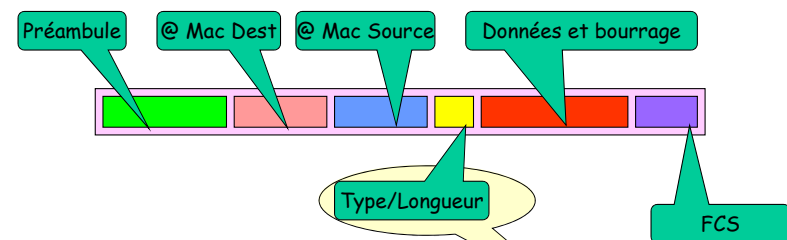
- ↳ Un Numéro de séquence pour savoir si c'est le premier ping, le second ou ...

- ↳ Enfin les données du ping, à savoir une suite d'octets.

```
> Internet Control Message Protocol
  Type: 8 (Echo (ping) request)
  Code: 0
  Checksum: 0xe06d [correct]
  [Checksum Status: Good]
  Identifier (BE): 7877 (0x1ec5)
  Identifier (LE): 50462 (0xc51e)
  Sequence number (BE): 1 (0x0001)
  Sequence number (LE): 256 (0x0100)
  [Response frame: 4]
  Timestamp from icmp data: Nov 2, 2022 14:25:59.000000000 Paris, Madrid
  [Timestamp from icmp data (relative): 0.599695008 seconds]
  > Data (48 bytes)
```



Résumé technique : format des trames Ethernet II



Préambule	8 octets
@ Mac Dest	6 octets
@ Mac Source	6 octets
Type / Longueur	2 octets
Données et bourrage	Taille variable
FCS	4 octets

Ethernet II : Type de données



Constitution d'une trame Ethernet



Le préambule



- Le préambule est une suite de 0 et de 1 alternés.
 - ↳ Il permet à l'horloge du récepteur de se synchroniser sur celle de l'émetteur. Comme la transmission est asynchrone, il est possible qu'une partie du préambule soit perdue (il existe aussi un paramètre des cartes réseau pour faire un préambule normal ou court).
- Les 8 octets du préambule se décomposent de la façon suivante :
 - ↳ **Le préambule** : 7 octets (10101010) soit 56 bits. Ils permettent aux autres stations de se synchroniser par rapport à la station émettrice.
 - ↳ **Le délimiteur de début de trame** (Start Frame Delimiter) : 1 octet (10101011). Il permet aux autres stations d'acquiescer la synchronisation du début de trame.



Constitution d'une trame Ethernet



Les adresses MAC



MAC = Medium Access Control

- Les adresses MAC identifient une machine dans le réseau. Celles-ci sont spécifiques à chaque carte réseau et doivent être uniques dans un réseau (en fait, elles sont uniques dans le monde).
- Elles sont constituées de 6 octets :
 - ↳ Les 3 premiers octets font référence au constructeur de l'interface. Ils sont uniques et sont attribués par l'IEEE.
 - ↳ Les 3 octets suivants donnent le numéro d'interface (de série) chez ce constructeur.

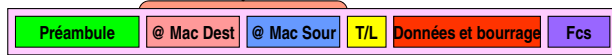
00:11:20:XX:XX:XX	Cisco
00:12:3F:XX:XX:XX	Dell
88:51:FB:XX:XX:XX	HP



Constitution d'une trame Ethernet



Les adresses MAC



- L'adresse source est toujours celle d'une interface unique (unicast).
- La destination peut être une adresse unique, de groupe (multicast, dans ce cas, le bit de poids faible de l'octet de poids fort est à 1) ou de diffusion générale (broadcast = FF:FF:FF:FF:FF:FF).



Constitution d'une trame Ethernet



Le champ longueur / type



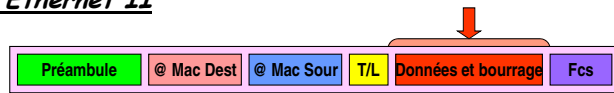
- Dans le standard Ethernet II (valeur du champ "type" supérieure à 1536, 0x0600), il indique le type de protocole suivant employé pour transporter le message. Les valeurs les plus courantes sont :

ARP	0x0806
IP	0x0800

Vous connaissez IP, ARP sera étudié en détail au semestre 2.



Les données - Ethernet II



- C'est le champ de données Ethernet, à savoir les données utiles.
 - ↳ La taille du champ de données doit être supérieure à 46 octets.
 - ↳ Dans la négative, une "séquence de bourrage" ou "padding" est rajoutée de manière à attendre la taille minimum d'octets à transmettre.



Le champ de contrôle



- Le FCS : *Frame Check Sequence* est un champ de 4 octets qui permet de valider l'intégrité de la trame à 1 bit près. Il utilise un CRC (*Cyclic Redundancy Check*) qui englobe tous les champs de la trame.
- Ainsi la station réceptrice peut vérifier si la trame est correcte et si c'est le cas, elle pourra l'exploiter.
- Il n'existe pas de champ pour coder la taille de la trame Ethernet.
 - ↳ La fin d'une trame est caractérisée par la fin de l'activité de la liaison série.
 - ↳ Les 4 octets qui précèdent représentent le FCS.
 - ↳ L'octet encore précédant est le dernier octet des données IPv4 (ou du bourrage).

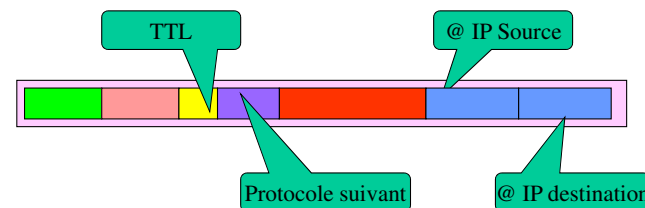


Sommaire

1. Constitution d'une trame Ethernet
2. Constitution d'un paquet IPv4
3. Notions d'encapsulation
4. Modèle en couches
5. Le modèle OSI
6. Le modèle TCP/IP
7. Comparaison des modèles et usages
8. Rôle de la couche transport
9. Les protocoles de la couche transport
10. Focus sur le protocole TCP
11. Exemple d'échange TCP



Résumé technique : entête IP les champs les plus importants



Remarque : la longueur standard d'une entête IPv4 est de 20 octets

```
Internet Protocol, Src: 192.168.1.47 (192.168.1.47), Dst: 192.168.1.254 (192.168.1.254)
  Version: 4
  Header length: 20 bytes
  Differentiated Services Field: 0x00 (DSCP 0x00: Default; ECN: 0x00)
  Total Length: 60
  Identification: 0x0747 (1863)
  Flags: 0x00
  Fragment offset: 0
  Time to live: 128
  Protocol: ICMP (1)
  Header checksum: 0x0000 [validation disabled]
  Source: 192.168.1.47 (192.168.1.47)
  Destination: 192.168.1.254 (192.168.1.254)
```



Sommaire

1. Constitution d'une trame Ethernet
2. Constitution d'un paquet IPv4
3. **Notions d'encapsulation**
4. Modèle en couches
5. Le modèle OSI
6. Le modèle TCP/IP
7. Comparaison des modèles et usages
8. Rôle de la couche transport
9. Les protocoles de la couche transport
10. Focus sur le protocole TCP
11. Exemple d'échange TCP



Définition

- On appelle **encapsulation** les processus qui permettent de construire la trame à partir des données de l'utilisateur.
- Sur notre exemple, le ping généré par l'utilisateur ne peut pas être envoyé tel quel à sa destination :
 - ↳ Le ping est contenu dans un message ICMP.
- La machine émettrice rajoute toutes les entêtes vus précédemment :
 - ↳ L'entête IP
 - ↳ L'entête Ethernet.
 - ↳ Puis calcule le FCS.
 - ↳ Enfin rajoute le préambule.
- La trame Ethernet est ainsi constituée et peut être émise sur le support de transmission.



Définition

- On appelle **désencapsulation** les processus inverses.
- Sur notre exemple, la machine réceptrice va :
 - ↳ Détecter l'arrivée de la trame Ethernet.
 - ↳ Vérifier si celle-ci est correcte.
 - ↳ Puis va lire et retirer petit à petit les entêtes (dans l'ordre, l'entête ethernet, puis IPv4 pour récupérer le message ICMP contenant le ping).

```
> Frame 4: 98 bytes on wire (784 bits), 98 bytes captured (784 bits) on interface eth0, id 0
> Ethernet II, Src: Dell_69:f2:10 (64:00:6a:69:f2:10), Dst: HewlettP_14:72:58 (d4:85:64:14:72:58)
> Internet Protocol Version 4, Src: 172.16.1.254, Dst: 172.16.1.22
  0100 .... = Version: 4
  .... 0101 = Header Length: 20 bytes (5)
> Differentiated Services Field: 0x00 (DSCP: CS0, ECN: Not-ECT)
  Total Length: 84
  Identification: 0x12d9 (4825)
> Flags: 0x0000
  Fragment offset: 0
  Time to live: 64
  Protocol: ICMP (1)
  Header checksum: 0x0c9c [validation disabled]
  [Header checksum status: Unverified]
  Source: 172.16.1.254
  Destination: 172.16.1.22
> Internet Control Message Protocol
```



Quizz

En résumé sur la trame 4 (diapo 23), repérons les champs suivants :

Dans l'entête Ethernet :

@Mac source : 64:00:6a:69:f2:10
@Mac destination : D4:85:64:17:72:58
Protocole qui suit : IPv4 (0x0800)

La lecture de la trame Ethernet permet d'en déduire que le premier octet des données Ethernet est le premier octet d'une entête IP

L'entête IPv4 :

@IP source : 172.16.1.254
@IP destination : 172.16.1.22
Protocole qui suit : Protocole qui suit : ICMP (valeur du champ : 1)
TTL : 64





Que transporte le paquet IPv4 ?

➤ Exemple de capture :

```
Frame 16: 77 bytes on wire (616 bits), 77 bytes captured (616 bits) on interface 0
Ethernet II, Src: Compalin_e1:83:90 (1c:75:08:e1:83:90), Dst: f4:ca:e5:48:79:f9 (f4:ca:e5:48:79:f9)
Internet Protocol, Src: 192.168.1.47 (192.168.1.47), Dst: 192.168.1.254 (192.168.1.254)
Version: 4
Header length: 20 bytes
Differentiated Services Field: 0x00 (DSCP 0x00: Default; ECN: 0x00)
Total Length: 63
Identification: 0x074e (1870)
Flags: 0x00
Fragment offset: 0
Time to live: 128
Protocol: UDP (17)
Header checksum: 0x0000 [validation disabled]
Source: 192.168.1.47 (192.168.1.47)
Destination: 192.168.1.254 (192.168.1.254)
User Datagram Protocol, Src Port: 54998 (54998), Dst Port: domain (53)
Domain Name System (query)
```

➤ Repérons dans l'entête IPv4 le protocole qui va suivre :

↳ Le champ "protocol" contient la valeur 17 > le protocole suivant est UDP.

➤ Les données importantes contenues dans l'entête UDP sont indiquées dans le résumé wireshark -> ce sont les ports sources (54998) et destination (53).

➤ Les données transportées dans ce datagramme UDP représentent une question au DNS. Wireshark affiche cela car un des 2 ports (ici le destination) est le port 53 et Wireshark sait que le port 53 correspond au protocole DNS.



Que transporte le paquet IPv4 ?

➤ Exemple de capture :

```
Frame 26: 101 bytes on wire (808 bits), 101 bytes captured (808 bits) on interface 0
Ethernet II, Src: f4:ca:e5:48:79:f9 (f4:ca:e5:48:79:f9), Dst: Compalin_e1:83:90 (1c:75:08:e1:83:90)
Internet Protocol, Src: 63.245.216.132 (63.245.216.132), Dst: 192.168.1.47 (192.168.1.47)
Version: 4
Header length: 20 bytes
Differentiated Services Field: 0x00 (DSCP 0x00: Default; ECN: 0x00)
Total Length: 87
Identification: 0x0196 (406)
Flags: 0x02 (Don't Fragment)
Fragment offset: 0
Time to live: 236
Protocol: TCP (6)
Header checksum: 0xb2b9 [validation disabled]
Source: 63.245.216.132 (63.245.216.132)
Destination: 192.168.1.47 (192.168.1.47)
Transmission Control Protocol, Src Port: https (443), Dst Port: 49368 (49368), Seq: 3760, Ack: 526, Len: 47
Secure Socket Layer
```

➤ Repérons dans l'entête IPv4 le protocole qui va suivre :

↳ Le champ "protocol" contient la valeur 6 > le protocole suivant est TCP.

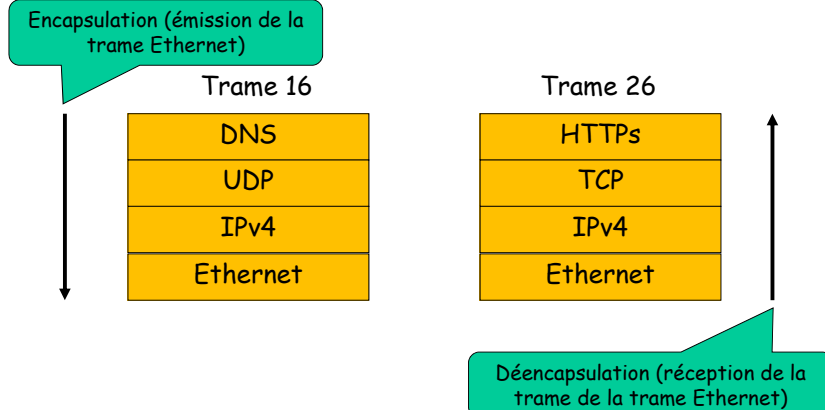
➤ Les données importantes contenues dans l'entête TCP sont indiquées dans le résumé wireshark -> ce sont les ports sources (443) et destination (49368).

➤ Les données transportées dans ce datagramme TCP représentent un message du protocole HTTPS. Wireshark affiche cela car un des 2 ports (ici le source) est le port 443 et Wireshark sait que le port 443 correspond au protocole HTTPS.



Challenge

➤ Dessinez la pile de protocoles (*protocol stack*) des trames 16 et 26.



Sommaire

1. Constitution d'une trame Ethernet
2. Constitution d'un paquet IPv4
3. Notions d'encapsulation
4. **Modèle en couches**
5. Le modèle OSI
6. Le modèle TCP/IP
7. Comparaison des modèles et usages
8. Rôle de la couche transport
9. Les protocoles de la couche transport
10. Focus sur le protocole TCP
11. Exemple d'échange TCP



Pourquoi un découpage en couches ?

- De l'étude précédente, on peut en déduire que l'on fonctionne par "couches".
- Pourquoi ? Cela permet de réduire la complexité de conception.
 - La plupart des réseaux sont organisés en couches.
 - Le nombre, le contenu et la fonction des couches diffèrent selon les réseaux.
- Il y a la couche pour la trame Ethernet (composée de l'entête Ethernet et des données Ethernet).
- Il y a la couche pour le paquet IPv4 (composée de l'entête IPv4 et des données IPv4).
- Il y a la couche pour le segment TCP (composée de l'entête TCP et des données TCP).



Pourquoi un découpage en couches ?

Couche n+1

Couche n

- La couche n fournit des services à la couche supérieure.
- La couche n dissimule les détails d'implémentation à la couche n+1.
- Entre chaque paire de couches, il y a une interface.



Pourquoi un découpage en couches ?

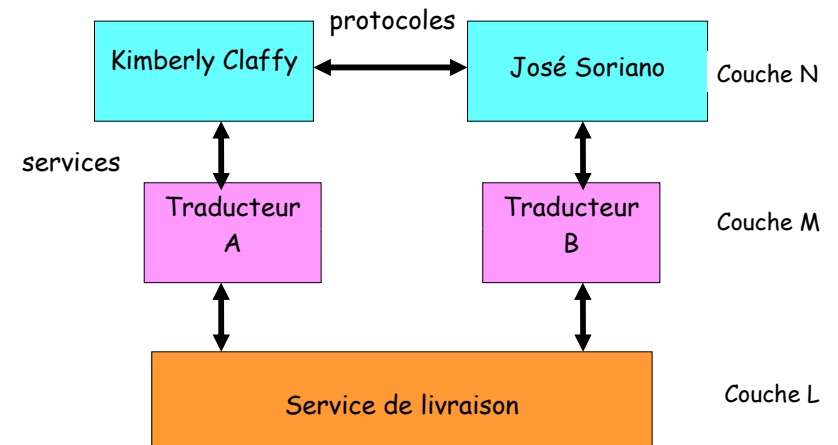
Couche n+1

Couche n

- La couche n peut être vue comme une machine virtuelle.
- L'interface entre 2 couches définit les opérations fondamentales et les services qu'offre la couche n à la couche n+1.
- Remarque importante** : on peut changer la nature de la couche n tant que l'on conserve les opérations et services offerts à la couche n+1.



Notions de services et protocoles





Déroulement de l'échange

- Kimberly envoie un programme en Python à son correspondant José pour relecture et optimisation (protocole = Python).
- Elle confie son programme et sa requête rédigée en anglais à son traducteur A.
 - ↳ Celui-ci la traduit en allemand c'est le service rendu (protocole = utilisation de l'allemand entre traducteurs).
- Le service de livraison assure la transmission.
- Le traducteur B reçoit le message.
 - ↳ Comprend la réponse (car l'allemand est le protocole utilisé entre traducteurs).
 - ↳ Traduit la réponse en espagnol (c'est le service rendu) et transmet le tout à José.



Déroulement de l'échange

- Remarques importantes :
 - ↳ Le protocole entre traducteurs peut changer sans perturber l'échange entre nos deux scientifiques.
 - ↳ Le service de livraison peut utiliser le support de son choix sans affecter l'échange.



Vocabulaire

- L, M, N sont les couches dans notre modèle.
- Mdestination communique avec Msource : la communication se fait d'égal à égal. Ce sont des couches **homologues**.
- Le protocole de couche M = règles utilisées par Msource pour communiquer avec Mdestination.
- **Définition générale :**
 - Un protocole est un ensemble de règles ou conventions qui détermine le format et la transmission des données.*
- Si la couche n d'un ordinateur communique avec la couche n d'un autre ordinateur, les règles et conventions de cette communication sont collectivement appelés **protocole de couche n**.
- Exemple : un protocole bien connu : IP Internet Protocol



Définition d'un service

- Il est défini par un ensemble de primitives (**actions simples**) qu'un processus utilisateur emploie pour accéder au service.
- Les primitives indiquent au service d'exécuter une action ou de rendre compte d'une action entreprise par un processus pair.
- Cet ensemble de primitives dépend de la nature du service fourni.



Exemples de primitives utilisées pour un service simple avec connexion

- Listen : état bloquant dans l'attente d'une connexion.
- Connect : établissement d'une connexion avec un processus pair à l'écoute.
- Receive : état bloquant dans l'attente d'un message entrant.
- Send : envoi un message au processus pair.
- Disconnect : libère une connexion.



Problèmes à gérer

- Mécanisme d'identification des processus émetteurs.
 - ↳ Recours à l'adressage.
- Règles de transfert des données :
 - ↳ Dans 1 ou 2 directions.
 - ↳ Nombre de canaux logiques et priorité.
- Contrôle d'erreur.
- Contrôle de flux.
- Transmission de messages de longueur variable.



Sommaire

1. Constitution d'une trame Ethernet
2. Constitution d'un paquet IPv4
3. Notions d'encapsulation
4. Modèle en couches
5. **Le modèle OSI**
6. Le modèle TCP/IP
7. Comparaison des modèles et usages
8. Rôle de la couche transport
9. Les protocoles de la couche transport
10. Focus sur le protocole TCP
11. Exemple d'échange TCP



Présentation

- Années 80 : L'ISO, l'organisation internationale de normalisation examine différentes structures de réseaux existantes afin d'en dégager un ensemble de règles.
- En 1984, à la suite de ces recherches, publication du modèle OSI
 - Open Systems Interconnection*
 - Interconnexion de Systèmes Ouverts*
- *But* : aider les fournisseurs à créer des réseaux compatibles avec d'autres réseaux.



Le modèle OSI



Présentation

- Il réduit la complexité.
- Il uniformise les interfaces.
- Il facilite la conception modulaire.
- Il assure l'interopérabilité entre technologies.
- Il doit donc contribuer à faciliter les évolutions et l'acquisition des connaissances et simplifier la maintenance des réseaux.



Le modèle OSI



La couche 1

- Rôle : transmettre des bits à l'état brut sur un canal de communication
- les problèmes de conception concernent les interfaces (mécaniques et électriques), la synchronisation et le support physique de transmission
- Unité d'encapsulation ou regroupement logique : **bit** / **bit**
- Exemples d'unités fonctionnant à ce niveau : carte réseau (connecteurs physiques), media, répéteur (*repeater*), concentrateur (*hub*).



Le modèle OSI



La couche 2

- Rôle : Accès au média
- Ajoute un entête aux informations de couches supérieures
- Cet entête contient en autres l'adresse matérielle de l'unité destination ou de l'unité suivante.
- Elle est divisée en deux sous-couches :
 - LLC : Logical link Control
 - MAC : Media access control
- Unité d'encapsulation ou regroupement logique : **trame** / **frame**
- Exemples d'unités fonctionnant à ce niveau : carte réseau (LLC et MAC), pont (*bridge*), commutateur (*switch*).



Le modèle OSI



La couche 3

- Adresses Réseau/Hôte
- rôle : sélection du meilleur chemin sur un interréseau : routage
- s'adapte aux capacités de la couche 2, gère la fragmentation
- Ajoute un entête aux informations de couches supérieures
- Cet entête contient entre autres l'adresse logique de l'unité destination
- Unité d'encapsulation ou regroupement logique : **paquet** / **packet**
- Exemples d'unités fonctionnant à ce niveau : routeurs.



Le modèle OSI



La couche 4

- Connexions de bout en bout
- Rôle : Segmentation et réassemblage des données dans le bon ordre. Etablissement et fermeture de circuits virtuels orientés connexion
- Est capable d'assurer la livraison de paquets avec correction d'erreurs, reprise sur incident et contrôle de flux.
- Unité d'encapsulation ou regroupement logique : **segment** / **datagram**
- Exemples d'unités fonctionnant à ce niveau : routeurs



Le modèle OSI



La couche 5

- Communications interhôtes
- Rôle : établir, gérer et fermer les connexions entre applications
- Elle offre plusieurs services comme par exemple :
 - la gestion du dialogue
 - la gestion du jeton
 - la synchronisation
- Unité d'encapsulation ou regroupement logique : **donnée** / **data**
- Exemples d'unités fonctionnant à ce niveau : logiciels



Le modèle OSI



La couche 6

- Rôle : Représentation des données :
 - ↳ Codage.
 - ↳ Services de transfert de syntaxe.
 - ↳ Services de conversion.
 - ↳ Services de cryptage.
 - ↳ Services de compression.
- Unité d'encapsulation ou regroupement logique : **donnée** / **data**
- Exemples d'unités fonctionnant à ce niveau : logiciels.



Le modèle OSI



La couche 7

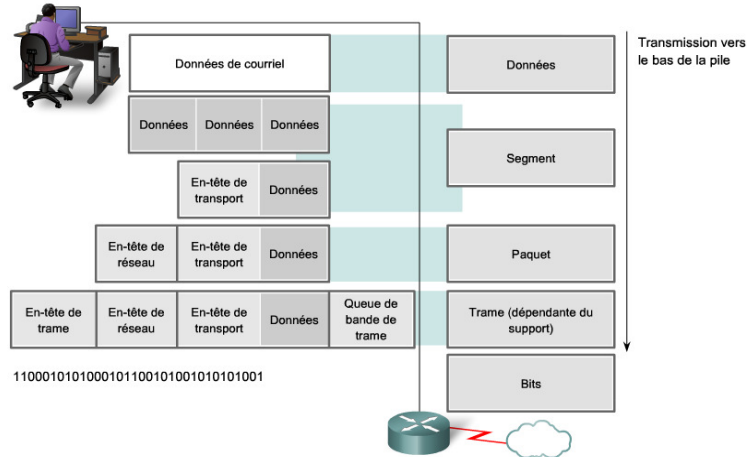
- Rôle : fournir des services réseau aux applications de l'utilisateur :
 - ↳ Services de fichiers.
 - ↳ Services d'impression.
 - ↳ Services de messagerie.
 - ↳ Services de bases de données.
- Elle détermine la disponibilité des ressources entre deux nœuds.
- Unité d'encapsulation ou regroupement logique : **donnée** / **data**
- Exemples d'unités fonctionnant à ce niveau : logiciels.



Le modèle OSI



Exemple d'encapsulation de données



Encapsulation, couches et modèle OSI



Sommaire

1. Constitution d'une trame Ethernet
2. Constitution d'un paquet IPv4
3. Notions d'encapsulation
4. Modèle en couches
5. Le modèle OSI
6. Le modèle TCP/IP
7. Comparaison des modèles et usage
8. Rôle de la couche transport
9. Les protocoles de la couche transport
10. Focus sur le protocole TCP
11. Exemple d'échange TCP

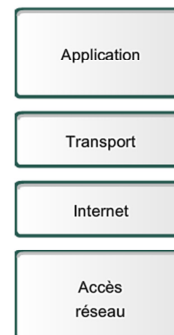


Le modèle TCP/IP



Présentation

- C'est le modèle utilisé par l'Arpanet et son descendant l'Internet.
- Il est conçu dans le but de :
 - ↳ Maintenir la disponibilité du système quoiqu'il arrive.
 - ↳ Pouvoir utiliser des applications très diversifiées.
 - ↳ Pouvoir utiliser les infrastructures réseaux existantes.



Le modèle TCP/IP



La couche *Accès au réseau*

- Cette couche n'est pas définie par TCP/IP.
- TCP/IP utilise le protocole du réseau local correspondant.
- L'hôte doit se connecter au réseau pour pouvoir envoyer des paquets.
 - ↳ Autre nom : couche hôte-réseau.



La couche Internet

- TCP/IP définit un réseau à commutation de paquets fondé sur une couche d'interconnexion de réseaux en mode non connecté.
- Elle définit un format de paquet et un protocole nommé IP *Internet protocol*.
- Son rôle est :
 - D'acheminer les paquets IP jusqu'à leur destination (routage).
 - D'adapter la taille des paquets aux caractéristiques de la couche accès réseau (fragmentation).
- Elle s'apparente à la couche réseau du modèle OSI sur le plan fonctionnel.



La couche Transport

- Son rôle est de :
 - ↳ Permettre à des entités paires sur les hôtes source et destination de mener une conversation.
- Deux protocoles ont été définis :
 - ↳ TCP *Transmission Control Protocol*.
 - ↳ UDP *User Datagram Protocol*.



La couche transport : le protocole TCP

- Protocole fiable avec connexion.
- Il garantit la livraison sans erreur d'un flot d'octets.
 - ↳ Au départ l'expéditeur (la couche Transport) scinde le flot d'octets en segments qu'il passe à la couche Internet.
 - ↳ A l'arrivée, le destinataire (la couche transport) réassemble les segments reçus.
- Les processus pairs voient un flux continu de données.
- TCP assure :
 - ↳ Un contrôle de flux. L'émetteur s'assure que le récepteur a la capacité de recevoir les données envoyées par l'émetteur.
 - ↳ La retransmission des segments erronés ou perdus.
 - ↳ La remise en ordre des segments reçu. En effet, les paquets IP ne passant pas forcément par le même chemin, rien ne dit qu'ils sont reçus dans l'ordre d'émission.



La couche transport : le protocole UDP

- Protocole non fiable sans connexion
- Laisse aux applications la charge d'assurer elles-mêmes le séquençement et le contrôle de flux si nécessaire.
- Utilisé par les applications type demande-réponse (environnement client-serveur).
- Utilisé pour transmettre des données à courte durée de vie
 - ↳ Par exemple : l'heure.



Le modèle TCP/IP



La couche Application

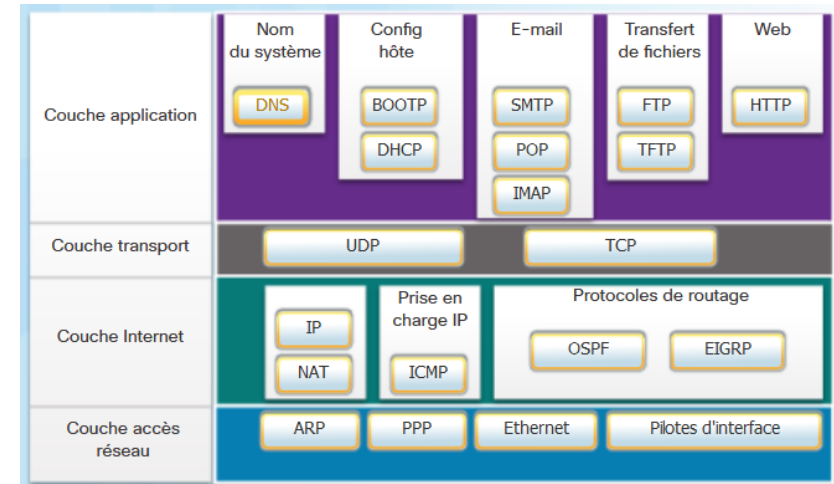
- Le modèle ne distingue pas les fonctionnalités des couches session et présentation.
- Chaque application gère ces fonctionnalités en fonction de ses besoins.
- Cette couche a défini de nombreux services et protocoles de haut niveau.



Le modèle TCP/IP



Pile de protocoles



Encapsulation, couches et modèle OSI



Sommaire

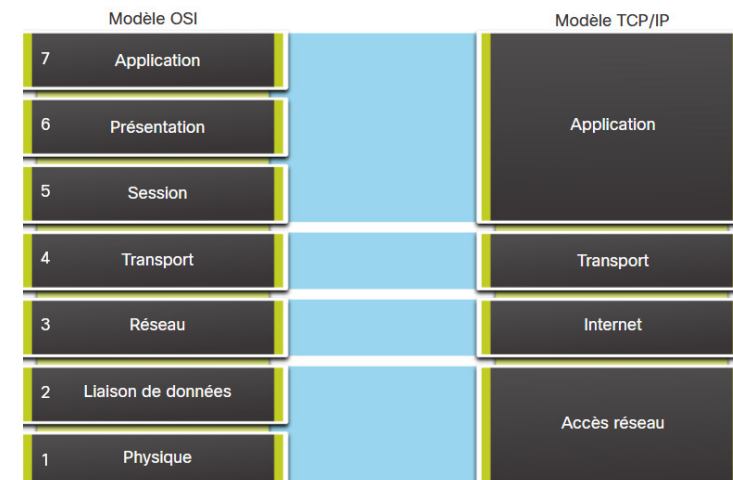
1. Constitution d'une trame Ethernet
2. Constitution d'un paquet IPv4
3. Notions d'encapsulation
4. Modèle en couches
5. Le modèle OSI
6. Le modèle TCP/IP
7. Comparaison des modèles et usages
8. Rôle de la couche transport
9. Les protocoles de la couche transport
10. Focus sur le protocole TCP
11. Exemple d'échange TCP



Comparaison des modèles et usages



Comparaison des deux modèles





Comparaison des deux modèles

- TCP/IP ne définit que des protocoles. Les interfaces ne sont pas normalisées.
- TCP/IP n'est pas un modèle général : il décrit uniquement la pile de protocoles TCP/IP.
- Il ne définit pas les couches physiques et liaison de données.
- Bien que peu normalisé au regard de l'OSI, le pragmatisme de sa conception et donc son efficacité l'a rendu modèle dominant dans le monde des réseaux malgré un certain nombre de lacunes.



Usage des modèles

- Bilan :
 - ↳ Les organisations mondiales ont mis trop de temps à publier le modèle OSI. De ce fait, un autre modèle a émergé : le modèle TCP/IP.
- Ce dernier n'est cependant pas satisfaisant car certaines couches ne sont pas bien définies.
- Pour les techniciens réseau que nous sommes un modèle intermédiaire serait l'idéal.



Usage des modèles

- Voici un modèle utilisé en maintenance et dépannage.
- Il est composé de 5 couches :

Couche 5 = Application TCP/IP
Couche 4 = Couche Transport OSI
Couche 3 = Couche Réseau OSI
Couche 2 = Couche liaison de données OSI
Couche 1 = Couche physique OSI



Usage des modèles

- Les couches 1 à 4 représentent bien le fonctionnement des différents équipements réseaux :
 - ↳ Serveur (port 80 d'un serveur Web) : couche 4
 - ↳ Routeur : Couche 3
 - ↳ Switch : Couche 2
 - ↳ Câble : Couche 1
- Couche Application TCP/IP : Logiciels



Usage des modèles

➤ On effectuera donc le dépannage (troubleshooting) en partant des couches basses (couche 1) aux couches hautes (logiciels).

➤ Exemple :

- ↳ Vous êtes sur un poste de travail en salle Bât 10-002. Le PC client dispose de 3 interfaces réseau et est démarré sous Windows.
- ↳ Vous lancez le logiciel "putty" pour prendre la main à distance via SSH (Secure Shell) sur un serveur situé sur un autre réseau.
- ↳ Info : Les câbles et prises ont été préalablement vérifiés par l'équipe technique.



Démarche efficace de dépannage

Etape 1 : Vérifier le câblage.

(couche 1) L'interface réseau du PC est-elle active ?

Est-elle connectée au switch local ?

Le switch local est-il connecté au switch de la baie de brassage ?

Etape 2 : Le switch local est-il fonctionnel ?

(couche 2) Vérifier que la configuration est vierge (routeur) ou standard (switch).

Dans la négative, corriger le problème.

Etape 3 : Le client peut-il faire un ping vers le serveur ?

(couche 3) Si non, vérifier la table de routage et corriger.

Etape 4 : le ping est fonctionnel mais le serveur ne répond pas par ssh.

(couche 4) Demander à votre encadrant de vérifier si le service SSH est lancé sur la machine serveur.



Sommaire

1. Constitution d'une trame Ethernet
2. Constitution d'un paquet IPv4
3. Notions d'encapsulation
4. Modèle en couches
5. Le modèle OSI
6. Le modèle TCP/IP
7. Comparaison des modèles et usages
8. Rôle de la couche transport
9. Les protocoles de la couche transport
10. Focus sur le protocole TCP
11. Exemple d'échange TCP



Rôle de la couche transport

➤ Reconstitution des segments :

↳ L'hôte recevant les blocs de données peut les diriger vers l'application appropriée (en utilisant principalement le numéro de port de destination).

↳ Il faut en outre que ces blocs de données individuels puissent être réassemblés dans un flux de données complet utile à la couche application.

↳ Les protocoles intervenant au niveau de la couche transport gèrent la façon dont les informations d'en-tête de la couche transport servent à réassembler les blocs de données flux qui seront transmis à la couche application.





Rôle de la couche transport

- Identification des applications:
 - ↳ Pour que les flux de données atteignent les applications auxquelles ils sont destinés, la couche transport doit identifier l'application cible. Pour cela, la couche transport affecte un identificateur à chaque application.
 - ↳ Les protocoles TCP/IP appellent cet identificateur un **numéro de port**. Chaque processus logiciel ayant besoin d'accéder au réseau se voit affecter un numéro de port unique sur son hôte. Ce numéro de port est inclus dans l'en-tête de la couche transport afin de préciser à quelle application ce bloc de données est associé.



Rôle de la couche transport en résumé

- La couche transport fait le lien entre la couche application et la couche inférieure responsable de la transmission réseau. Cette couche accepte les données provenant de plusieurs conversations et les fait descendre vers les couches inférieures sous forme de blocs faciles à gérer pouvant au final faire l'objet d'un multiplexage sur le support.
- Les applications n'ont pas besoin de connaître les détails du fonctionnement du réseau utilisé.
- Les couches inférieures ignorent que plusieurs applications envoient des données sur le réseau. Leur responsabilité se limite à livrer les données au périphérique approprié. La couche transport trie ensuite ces blocs avant de les acheminer vers l'application voulue.
- Le choix du protocole de la couche transport se fera en fonction de la nature des données à transporter.



Sommaire

1. Constitution d'une trame Ethernet
2. Constitution d'un paquet IPv4
3. Notions d'encapsulation
4. Modèle en couches
5. Le modèle OSI
6. Le modèle TCP/IP
7. Comparaison des modèles et usages
8. Rôle de la couche transport
9. Les protocoles de la couche transport
10. Focus sur le protocole TCP
11. Exemple d'échange TCP

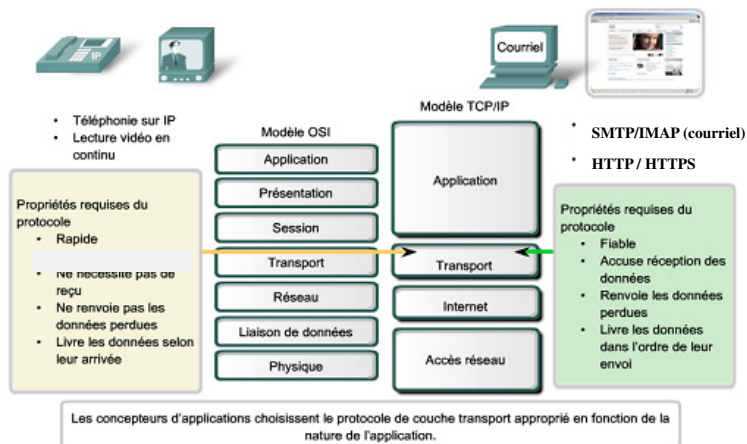


Protocoles utilisés

- Les deux protocoles de la suite de protocoles TCP/IP les plus couramment employés sont le protocole TCP (Transmission Control Protocol) et le protocole UDP (User Datagram Protocol).
- Ces deux protocoles gèrent les communications de nombreuses applications. Ce sont les fonctions spécifiques implémentées par chaque protocole qui les différencient.



Protocoles utilisés :



Le protocole UDP

- Le protocole UDP est un protocole simple, sans connexion, décrit par le document RFC 768. Il présente l'avantage d'imposer **peu de surcharge** pour l'acheminement des données. Les blocs de communication utilisés dans le protocole UDP sont appelés des datagrammes. Ces datagrammes sont envoyés « **au mieux** » par ce protocole de couche transport.
- Le protocole UDP est notamment utilisé par les applications suivantes :
 - Système de noms de domaine (DNS).
 - Lecture vidéo en continu.
 - Téléphonie sur IP (ToIP).
 - Service de mise à l'heure (NTP).
 - Attribution dynamique d'adresses IP (DHCP).

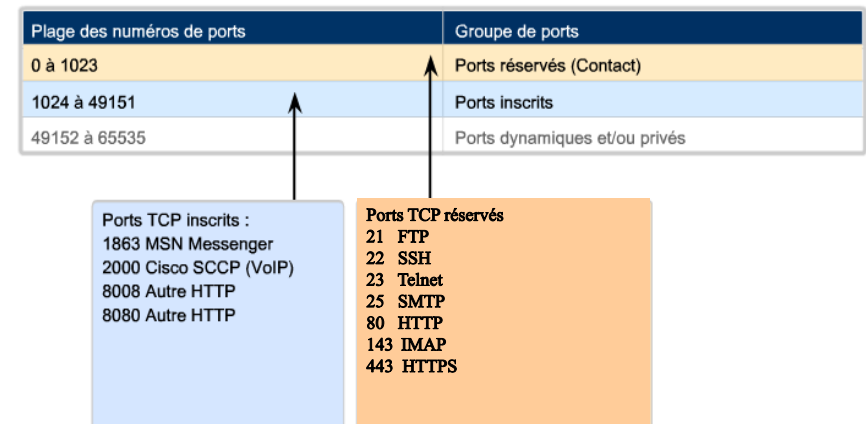


Le protocole TCP

- Le protocole TCP (Transmission Control Protocol) est un protocole avec connexion décrit dans le document RFC 793. Le protocole TCP **impose une surcharge** pour accroître les fonctionnalités. Le protocole TCP spécifie d'autres fonctions, à savoir la **livraison dans l'ordre**, l'**acheminement fiable** et le **contrôle de flux**.
- Le protocole TCP est utilisé par les applications suivantes :
 - Navigateurs Web.
 - Courriel.
 - Transferts de fichiers.
 - Prise en main à distance.

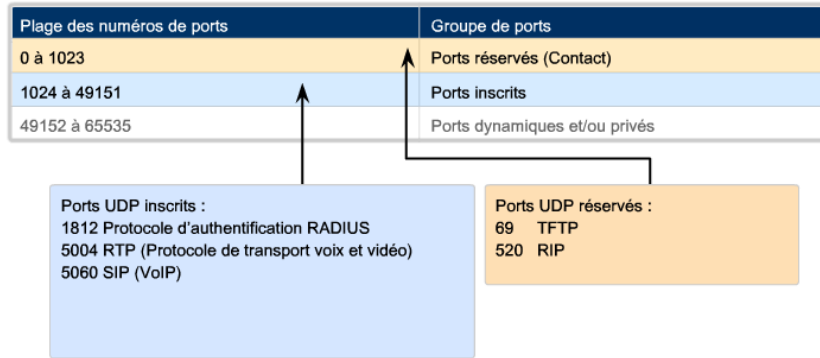


Les ports TCP





Les ports UDP



Les ports mixtes

- Les ports mixtes sont utilisés par TCP et UDP.
 - ↳ UDP sera utilisé pour un échange client ↔ serveur
 - ↳ TCP sera utilisé pour un échange serveur ↔ serveur
- On peut citer :

Pour les ports réservés :

53 DNS

Pour les ports inscrits :

5060 SIP



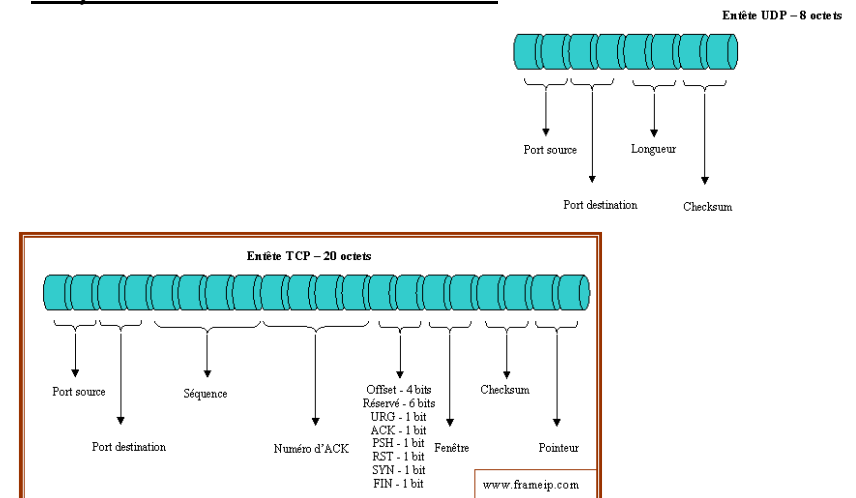
Etat des lieux de l'utilisation des ports un PC Windows

```
C:\Users\prof>netstat -an
Connexions actives

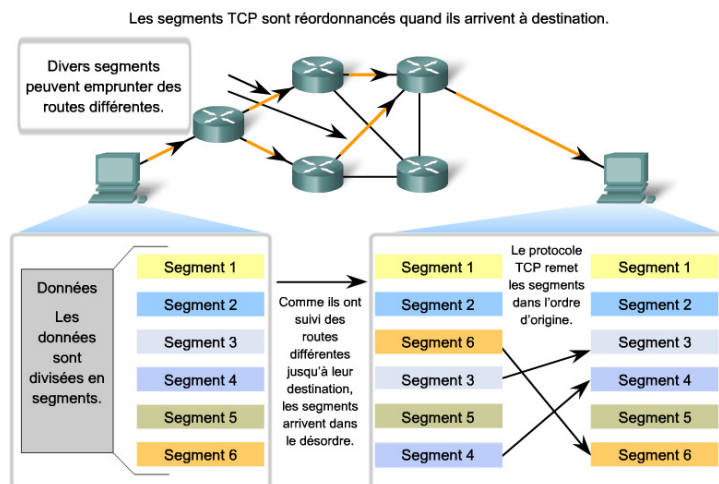
Proto Adresse locale Adresse distante état
TCP 0.0.0.0:135 0.0.0.0:0 LISTENING
TCP 0.0.0.0:445 0.0.0.0:0 LISTENING
TCP 0.0.0.0:3389 0.0.0.0:0 LISTENING
TCP 0.0.0.0:5357 0.0.0.0:0 LISTENING
TCP 0.0.0.0:49152 0.0.0.0:0 LISTENING
TCP 0.0.0.0:49153 0.0.0.0:0 LISTENING
TCP 0.0.0.0:49154 0.0.0.0:0 LISTENING
TCP 0.0.0.0:49155 0.0.0.0:0 LISTENING
TCP 0.0.0.0:49156 0.0.0.0:0 LISTENING
TCP 0.0.0.0:49160 0.0.0.0:0 LISTENING
TCP 127.0.0.1:49947 127.0.0.1:49948 ESTABLISHED
TCP 127.0.0.1:49948 127.0.0.1:49947 ESTABLISHED
TCP 192.168.1.39:139 0.0.0.0:0 LISTENING
TCP 192.168.1.39:50419 193.51.160.14:80 ESTABLISHED
TCP 192.168.1.39:50492 72.163.6.226:80 TIME_WAIT
TCP 192.168.1.39:50494 129.20.235.3:139 SYN_SENT
TCP [::]:135 [::]:0 LISTENING
TCP [::]:445 [::]:0 LISTENING
TCP [::]:3389 [::]:0 LISTENING
TCP [::]:5357 [::]:0 LISTENING
TCP [::]:49152 [::]:0 LISTENING
TCP [::]:49153 [::]:0 LISTENING
TCP [::]:49154 [::]:0 LISTENING
TCP [::]:49155 [::]:0 LISTENING
TCP [::]:49156 [::]:0 LISTENING
TCP [::]:49160 [::]:0 LISTENING
UDP 0.0.0.0:68 **
UDP 0.0.0.0:123 **
UDP 0.0.0.0:500 **
UDP 0.0.0.0:3000 **
```



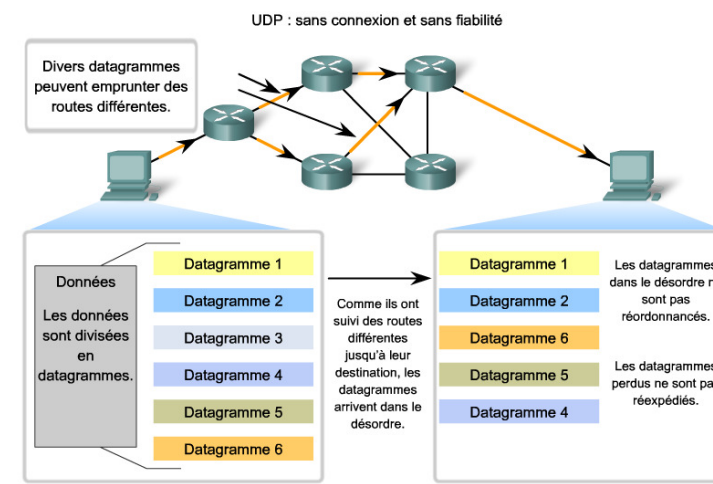
Comparaison des entêtes UDP et TCP



Comparaison rapide entre UDP et TCP



Comparaison rapide entre UDP et TCP



Sommaire

1. Constitution d'une trame Ethernet
2. Constitution d'un paquet IPv4
3. Notions d'encapsulation
4. Modèle en couches
5. Le modèle OSI
6. Le modèle TCP/IP
7. Comparaison des modèles et usages
8. Rôle de la couche transport
9. Les protocoles de la couche transport
10. Focus sur le protocole TCP
11. Exemple d'échange TCP

Principe de TCP

- Pour gérer la remise en ordre des segments TCP et la perte, TCP numérote chaque octets émis.
 - ↳ Avantage : on peut remettre en ordre très facilement les segments.
 - ↳ Avantage : on peut savoir si un segment est perdu (n'est pas reçu) et le réémettre.
 - ↳ Mais émettre le numéro de chaque octet émis sera beaucoup complexe. Donc on numérote bien chaque octet, mais on émet que le numéro du premier octet de chaque segment.
- Le numéro de séquence :
 - ↳ C'est le numéro du premier octet que l'on émet. Après émis du segment :
 - Soit le segment contenait des données utiles, dans ce cas le numéro de segment augmente.
 - Soit le segment ne contenait pas des données utiles, dans ce cas le numéro de segment ne change pas.
- Le numéro d'acknowledgment :
 - ↳ C'est le numéro du prochain octet que l'on espère recevoir (cela signifie que l'on a bien tout reçu les octets "jusqu'à numéro d'acknowledgment - 1").



Focus sur le protocole TCP



Points importants de l'entête TCP

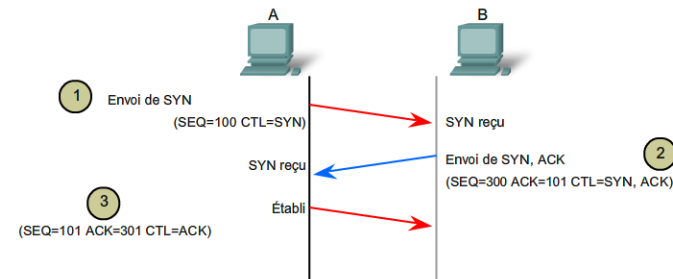
- Le numéro de séquence ou numéro d'ordre (sequence).
- Le numéro d'acquittement ou reçu (acknowledgement).
- Les drapeaux (flags) :
 - **sync** : Initier une connexion (ouverture cx TCP).
 - **fin** : Fermer la connexion TCP.
 - **rst** : Interrompre la connexion suite à une erreur..
 - **ack** : Acquitter la réception des données
 - **push** : Indiquer aux couches supérieures qu'elles peuvent utiliser le flux de données reconstitué.
 - **urg** : Indiquer une donnée urgente (peu utilisé à l'heure actuelle).
- L'émission d'un segment contenant un sync ou un fin fait :
 - ↳ Augmenter de 1 le numéro de séquence (lors de l'émission du flag).
 - ↳ Augmenter de 1 le numéro d'acquittement (lors de la réception du flag).



Focus sur le protocole TCP



Le protocole TCP - Ouverture de connexion



CTL = Indique quels bits de contrôle de l'en-tête TCP sont configurés sur 1.

- Le premier numéro de séquence étant aléatoire, c'est 3 échanges permettent de :
 - ↳ Synchroniser le numéro d'acquittement du récepteur au numéro de séquence de l'émetteur.
 - ↳ A l'émetteur d'avoir confirmation que le récepteur a bien reçu la bonne valeur du numéro de séquence de l'émetteur.

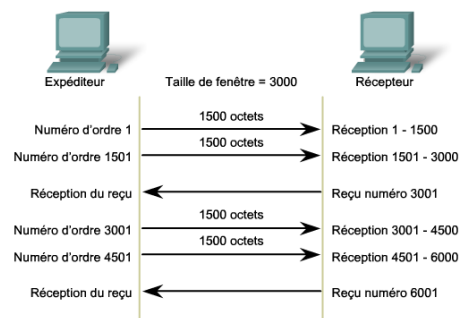


Focus sur le protocole TCP



Le protocole TCP - Echange de données

Reçu de segment TCP et taille de fenêtre



La **taille de fenêtre** détermine le nombre d'octets envoyés avant qu'un reçu ne soit attendu.
Le numéro du **reçu** est le même que le numéro du prochain octet attendu.

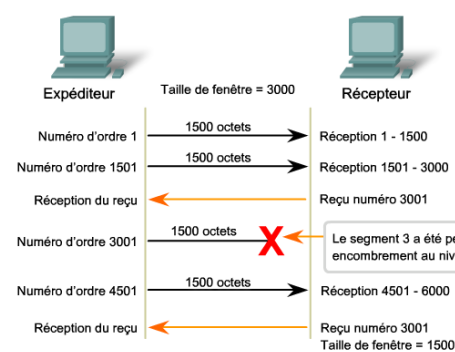


Focus sur le protocole TCP



Le protocole TCP - Echange de données

Encombrement TCP et contrôle de flux

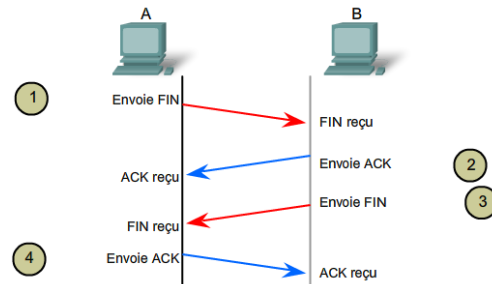


Si des segments sont perdus du fait d'un encombrement, le récepteur enverra un reçu pour le dernier segment séquentiel reçu et répondra en utilisant une taille de fenêtre réduite.





Le protocole TCP - Fermeture de connexion



Sommaire

1. Constitution d'une trame Ethernet
2. Constitution d'un paquet IPv4
3. Notions d'encapsulation
4. Modèle en couches
5. Le modèle OSI
6. Le modèle TCP/IP
7. Comparaison des modèles et usages
8. Rôle de la couche transport
9. Les protocoles de la couche transport
10. Focus sur le protocole TCP
11. Exemple d'échange TCP



Exemple d'échange TCP

